SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Name	Chennuri Sumedha
Register Number	1971260029
Course Code & Name	C Programming
Slot	C
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1 – Q3)
GitHub Repository URL	https://github.com/sumedha-2805/CSA0210.git
Submission Date	13-11-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — Searching a Library Catalogue

Task / ProblemStatement

The library storesbook accession numbers in an array (10 books now, 100,000 later). Implement linear search on the unsorted array and binary search on the sorted array, count the comparisons, and test with a book at the beginning, middle, end and a missing book.

Aim

Toimplement linear and binary search in C and compare the number of comparisons each one makes.

Concept / Theory

Linear search checks elements one by one from the start, so it can take up to n comparisons. Binary search works only on a sorted array: it compares the key with the middle element and discards half of the range each time, so it needs about logn comparisons.

2

Procedure / Algorithm

1. Linear search: check a[0], a[1], ... counting each comparison; return the index when found, else -1.
2. Binary search: set low = 0, high = n-1.
3. While low <= high: mid = low + (high - low)/2 and count one comparison.
4. If a[mid] == key return mid; if a[mid] < key set low = mid+1; else set high = mid-1.
5. If the range becomes empty, return -1.

Program / Source Code

```
#include <stdio.h>

intsteps; /* comparisons made by the last search */

/* Linear search on an unsorted array */
intlinear_search(const int a[], int n, int key)
{
    steps = 0;
    for (int i = 0; i < n; i++) {
        steps++;                        /* one comparison a[i] == key */
        if (a[i] == key)
            return i;
    }
    return -1;
}

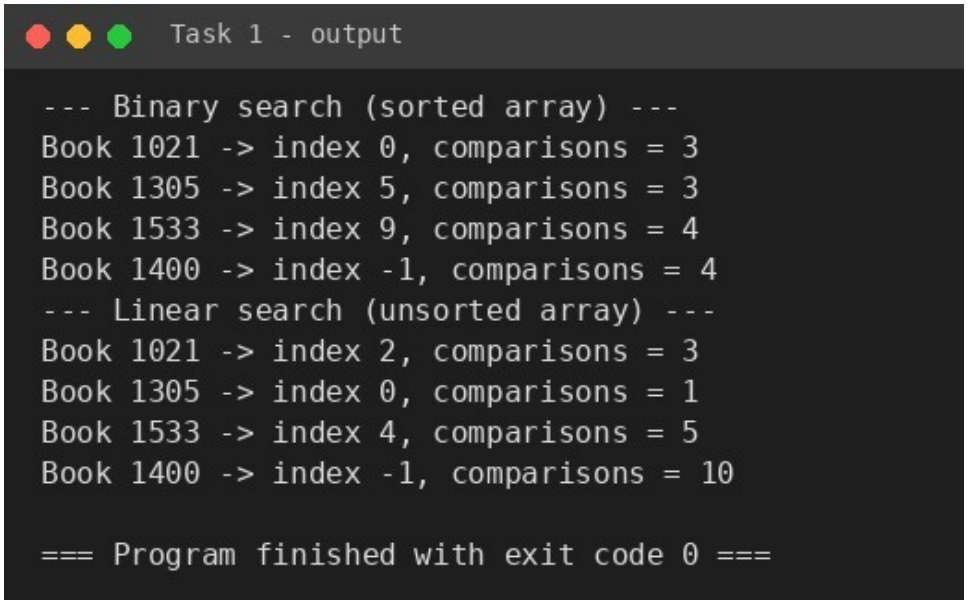
/* a[] is sorted in ascending order. Returns the index of key, or -1. */
intbinary_search(const int a[], int n, int key)
{
    steps = 0;
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;   /* no overflow */
        steps++;                            /* one comparison of a[mid] with key */
        if (a[mid] == key)
            return mid;
         else if (a[mid] < key)
             low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void)
{
```

```
    int accession[]={1021, 1048, 1103, 1190, 1222, 1305, 1377, 1402, 1460, 1533};
    int unsorted[]  ={1305, 1460, 1021, 1222, 1533, 1103, 1377, 1048, 1190, 1402};
    int n=  sizeofaccession / sizeof accession[0];
    int keys[]={1021,1305, 1533, 1400};

    printf("---Binarysearch (sorted array) ---\n");
    for (int k=0;k<4; k++) {
        int pos=binary_search(accession, n, keys[k]);
        printf("Book%d-> index %d, comparisons = %d\n", keys[k], pos, steps);
    }
    printf("---Linearsearch (unsorted array) ---\n");
    for (int k=0;k<4; k++) {
        int pos=linear_search(unsorted, n, keys[k]);
        printf("Book%d-> index %d, comparisons = %d\n", keys[k], pos, steps);
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

- Binarysearch results match the expected output: 1021 -> 3, 1305 -> 3, 1533 -> 4, missing 1400 -> 4 comparisons.
- Comparisons: linear best 1, worst n, average (n+1)/2. Binary best 1, worst floor(logn)+1.
- For 100,000 books: linear needs up to 100,000 comparisons; binary needs at most 17.
- Missing book needs 4 comparisons, the same as the worst successful search, because the range must shrink until it is empty.
- Binary search needs sorted data (otherwise discarding half is wrong). mid = low + (high-low)/2 is used so that low+high cannot overflow.
- If books are added daily: keeping the array sorted costs O(n) per insertion, so it is worth it only if searches are far more frequent than additions (or new accession numbers are added in increasing order).
- Alternatives: hash table O(1) average but extra space; interpolation search O(log log n) average, O(n) worst; binary search O(log n) with no extra space.

Result

Both searches were implemented and the program gave the expected output. Binary search is far faster than linear search for large catalogues.

TASK 2 — Sorting the Examination Hall Seating Register

Task / ProblemStatement

Implement bubble sort (withearly-exit flag), selection sort and insertion sort, count comparisons and swaps (or shifts), and test on sorted, nearly sorted, reverse-sorted and random arrays.

Aim

Toimplement three sorting algorithms and compare their comparisons and swaps for different input orders.

Concept / Theory

Bubble sort swaps neighbouring elements that are out of order. Selection sort picks the smallest remaining element and swaps it into place. Insertion sort inserts each element into the already sorted part by shifting

larger elementsright. All threeare $O(n^2)$ in the worst case.

Procedure / Algorithm

1. Bubblesort:repeat passes comparing a[j] and a[j+1], swap if out of order; stop if a pass has no swap.
2. Selection sort: for each i, find the minimum of a[i..n-1] and swap it with a[i].
3. Insertion sort: take key = a[i], shift larger elements of a[0..i-1] right, place key.
4. Count every comparison and every swap/shift.
5. Run on sorted, nearly sorted, reverse and random data for n = 100, 1,000, 10,000.

Program / Source Code

```
#include <stdio.h>
#include <stdlib.h>
long cmp, swp;

void bubble(int a[], int n) {
    cmp = swp = 0;
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;                        /* early-exit flag */
        for (int j = 0; j < n - 1 - i; j++) {
            cmp++;
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swp++; swapped = 1;
            }
        }
        if (!swapped) break;
}   }
void selection(int a[], int n) {

    cmp = swp = 0;
    for (int i = 0; i < n - 1; i++) {
        int m = i;
        for (int j = i + 1; j < n; j++) { cmp++; if (a[j] < a[m]) m = j; }
        if (m != i) { int t = a[i]; a[i] = a[m]; a[m] = t; swp++; }
}   }
void insertion(int a[], int n) {
                                                /* swp counts shifts */
    cmp = swp = 0;
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0) {
            cmp++;
            if (a[j] > key) { a[j + 1] = a[j]; swp++; j--; }
            else break;
        }
        a[j + 1] = key;
}   }
void fill(int a[], int n, int type) {

    for (int i = 0; i < n; i++) a[i] = i + 1;
```

```
    if (type == 1) for (int s = 0; s < n / 50 + 1; s++) {   /* nearly sorted */
        int x = rand() % n, y = rand() % n; int t = a[x]; a[x] = a[y]; a[y] = t; }
    if (type == 2) for (int i = 0; i < n; i++) a[i] = n - i; /* reverse */
    if (type == 3) for (int i = n - 1; i > 0; i--) {         /* random */
          int j = rand() % (i + 1); int t = a[i]; a[i] = a[j]; a[j] = t; }
}
int main(void){

    int demo[6] = {52, 31, 45, 12, 38, 27};
    int d[6]; for (int i = 0; i < 6; i++) d[i] = demo[i];
    bubble(d, 6);    printf("Demo6numbers    Bubble    cmp=%ld swaps=%ld  ->", cmp, swp);
    for (int i = 0; i < 6; i++) printf(" %d", d[i]); printf("\n");
    for (int i = 0; i < 6; i++) d[i] = demo[i];
    selection(d,6);printf("Demo6numbers      Selectioncmp=%ld  swaps=%ld  ->", cmp, swp);
    for (int i = 0; i < 6; i++) printf(" %d", d[i]); printf("\n");
    for (int i = 0; i < 6; i++) d[i] = demo[i];
    insertion(d,6);printf("Demo6numbers      Insertioncmp=%ld  shifts=%ld ->", cmp, swp);
    for (int i=0;i<6;i++)printf("%d",d[i]);printf("\n\n");

    const char *name[] = {"Sorted", "Nearly", "Reverse", "Random"};
    int sizes[] = {100, 1000, 10000};
    printf("%-6s%-8s|%-18s    |%-18s|%-18s\n","n","Order",      "Bubble cmp/swp", "Selection cmp/swp", "Insertion cmp/shift");
    srand(42);
    for (int s = 0; s < 3; s++) for (int t = 0; t < 4; t++) {

        int n = sizes[s];
        int *base=malloc(n*sizeof(int)),*a=malloc(n* sizeof(int));
        fill(base, n, t);
        long r[3][2];
        for (inti=0;i<n;i++)a[i]=base[i];bubble(a,            n);     r[0][0] = cmp; r[0][1] = swp;
        for (inti=0;i<n;i++)a[i]=base[i];selection(a,              n);r[1][0] = cmp; r[1][1] = swp;
        for (inti=0;i<n;i++)a[i]=base[i];insertion(a,              n);r[2][0] = cmp; r[2][1] = swp;
        printf("%-6d%-8s|%8ld/%-9ld|%8ld/%-9ld|%8ld/%-9ld\n",        n, name[t],
               r[0][0], r[0][1], r[1][0], r[1][1], r[2][0], r[2][1]);
        free(base); free(a);
    }
    return0;
}
```

Output Screenshot

```
Task 2 - output

Demo 6 numbers  Bubble    cmp=15 swaps=11  -> 12 27 31 38 45 52
Demo 6 numbers  Selection cmp=15 swaps=5   -> 12 27 31 38 45 52
Demo 6 numbers  Insertion cmp=14 shifts=11 -> 12 27 31 38 45 52

n      Order    | Bubble cmp/swp     | Selection cmp/swp  | Insertion cmp/shift
100    Sorted   |       99/0         |     4950/0         |       99/0
100    Nearly   |     3572/215       |     4950/3         |      314/215
100    Reverse  |     4950/4950      |     4950/50        |     4950/4950
100    Random   |     4814/2610      |     4950/96        |     2705/2610
1000   Sorted   |      999/0         |   499500/0         |      999/0
1000   Nearly   |   449414/14359     |   499500/21        |    15358/14359
1000   Reverse  |   499500/499500    |   499500/500       |   499500/499500
1000   Random   |   499290/254262    |   499500/992       |   255255/254262
10000  Sorted   |     9999/0         | 49995000/0         |     9999/0
10000  Nearly   | 49777530/1345373   | 49995000/201       |  1355372/1345373
10000  Reverse  | 49995000/49995000  | 49995000/5000      | 49995000/49995000
10000  Random   | 49994964/24733254  | 49995000/9994      | 24743244/24733254

=== Program finished with exit code 0 ===
```

Analysis / Discussion

- Early-exit flag: if a full pass makes no swap the array is already sorted, so bubble sort stops (sorted input needs only n-1 comparisons).
- Insertion sort is fast on nearly sorted data because each shift removes one out-of-order pair, so it does very little work when few pairs are out of order.
- Stable: bubble and insertion sort. Not stable: selection sort. Stability keeps students with equal marks in their original order.
- Best algorithm: sorted - bubble (flag) or insertion; nearly sorted - insertion; reverse - selection (fewest swaps); random - insertion (about half the comparisons).
- Unexpected: bubble sort on nearly sorted data was almost as slow as on random data, because a small element moves only one place left per pass.
- Conclusion: for a register sorted except 20 late entries, use insertion sort (almost linear time, stable).
- Alternative: counting sort takes O(n+k) time and O(k) space, but is not suitable when the range k is very large or keys are not integers.

Result

All three sorts were implemented and tested. Insertion sort is best for nearly sorted data, and the best choice depends on the input order.

TASK 3 — Merging Two Merit Lists

Task / ProblemStatement

Twocampuses givemeritlists sorted in descending order. Merge them into one descending list without sorting again, handling different lengths and equal scores.

Aim

Tomerge two sorted arrays into a third array in O(m + n) time.

Concept / Theory

Sinceboth lists are already sorted, the largest remaining score is always at the front of one list. Compare the two front elements, copy the larger into the output and move only that list forward. When one list ends, copy the rest of the other.

Procedure / Algorithm

1. Set i = 0, j = 0, k =0.
2. While both lists have elements: if a[i] >= b[j] copy a[i], else copy b[j]; advance that index and k.
3. Copy any remaining elements of a[].
4. Copy any remaining elements of b[].
5. Return k, the total count.

Program / Source Code

```
#include <stdio.h>
#include <stdlib.h>

/* a[] and b[] are merit lists sorted in DESCENDING order of score.
   Copies both into out[] in descending order and returns the total count. */
intmerge(const int a[], int na, const int b[], int nb, int out[])
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {           /* both lists have elements */
        if (a[i] >= b[j])                /* >= : on a tie, campus A comes first (stable) */
            out[k++] = a[i++];
        else
            out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];  /* copy what remains in a[] */
    while (j < nb) out[k++] = b[j++];  /* copy what remains in b[] */
    return k;
}

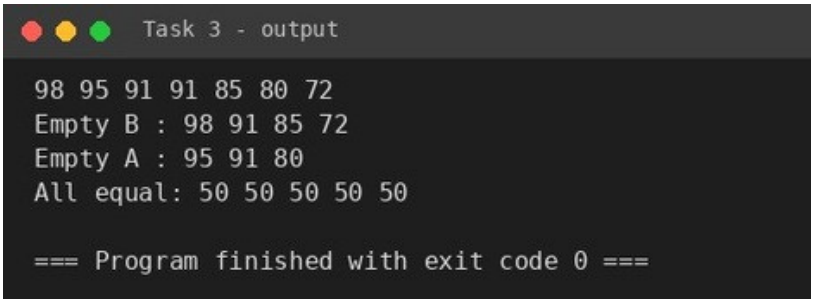
static void show(const int x[], int n)
{
    for (int i = 0; i < n; i++) printf("%d ", x[i]);
    printf("\n");
}

int main(void)
{
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];
    int n = merge(campusA, 4, campusB, 3, merged);
    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\n");

    /* extra tests */
    int e[1], r[3];
    int c1 = merge(campusA, 4, e, 0, merged);
    printf("Empty B : "); show(merged, c1);
```

```
    c1 = merge(e, 0, campusB, 3, r);
    printf("Empty A : "); show(r, c1);
    int eq1[] = {50, 50, 50}, eq2[] = {50, 50};
    int o[5];
    c1 = merge(eq1, 3, eq2, 2, o);
    printf("All equal: "); show(o, c1);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

- Output 98 95 91 91 85 80 72 matches the expected output.
- Empty list: the first loop does not run at all and the leftover loop copies the other list.
- Each element is compared at most once, because every comparison outputs one element and the indices never move back. Comparisons: best min(m,n), worst m+n-1.
- Equal scores keep campus order, because a[i] >= b[j] copies campus A first on a tie (stable).
- Merge needs about 2,000 / 20,000 comparisons for m = n = 1,000 / 10,000, while joining and sorting again needs about 21,900 / 285,800. The cost is extra memory of m+n elements.
- In-place merge is preferred when memory is limited, but it takes O(m·n) time if done simply.

Result

Themerge function was completed and gave the expected output; empty-list and equal-score tests also passed.

Test Cases / Validation Table

Recorddifferentinputs,expectedresults,actualresults, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 1: binary search, first book 1021	index 0, 3 comparisons	index 0, comparisons = 3	Pass
2	Task 1: binary search, middle book 1305	index 5, 3 comparisons	index 5, comparisons = 3	Pass
3	Task 1: binary search, last book 1533	index 9, 4 comparisons	index 9, comparisons = 4	Pass
4	Task 1: binary search, missing book 1400	index -1, 4 comparisons	index -1, comparisons = 4	Pass
5	Task 1: linear search (unsorted), missing book 1400	index -1, 10 comparisons	index -1, comparisons = 10	
6	Task 2: 52 31 45 12 38 27 with all three sorts	(n)	12 27 31 38 45 52 (all three)	Pass
7	Task 2: sorted input, n = 1,000, bubble sort with flag	12 27 31 38 45 52	9	Pass
8	Task 3: campus A {98,91,85,72}, campus B {95,91,80}	999 comparisons, 0 swaps	998 99 5/ 0 91 91 85 80 72	Pass
9	Task 3: one list empty (B empty)	98 95 91 91 85 80 72	98 91 85 72	Pass
10	Task 3: all scores equal {50,50,50} and {50,50}	98 91 85 72 50 50 50 50 50	50 50 50 50 50	Pass

Final Submission Checklist

- [ ] Student and course details completed
- [ ] Task statement, aim, theory, and algorithm included
- [ ] Program code is readable and runs successfully
- [ ] Output screenshot included
- [ ] Analysis and result written in own words
- [ ] Test cases completed where applicable
- [ ] GitHub repository link added if required

TASK 4 - Two-Dimensional Arrays: District Rainfall Records

Task / Problem Statement

Monthly rainfall(mm) of38districts for 12 months is stored in rain[38][12]. Find the annual total per district, the monthly average, the wettest district-month, and the districts whose rainfall is below 60% of the state average.

Aim

Tofilland process a 2-D array using separate functions, check them on a small 3 x 4 array, and compare row-wise and column-wise traversal time.

Concept / Theory

Inrain[i][j], i is thedistrict (row) and j is the month (column). C stores a 2-D array row by row, so rain[i][j] sits right next to rain[i][j+1]. Address = base + (i x 12 + j) x size. Going row-wise reads neighbouring memory (fast, cache friendly); going column-wise jumps 12 elements each step (slower).

Procedure / Algorithm

1

2 Declarerain[38][12] andfill it with a formula.

3 row_totals: add the 12 months of each district.

4 col_avgs: add the 38 districts of each month and divide by 38.

5 max_elem: scan all cells, remember the biggest value and its position.
State average = mean of the row totals; a district is deficient if its total < 0.6 x state average.

Program / Source Code

```
#include <stdio.h>
#include <time.h>
#define M 12

voidrow_totals(double a[][M], int d, int m, double t[]) {
    for (int i = 0; i < d; i++) {
        t[i] = 0;
        for (int j = 0; j < m; j++) t[i] += a[i][j];
    }
}

voidcol_avgs(double a[][M], int d, int m, double avg[]) {
    for (int j = 0; j < m; j++) {
        avg[j] = 0;
        for (int i = 0; i < d; i++) avg[j] += a[i][j];
        avg[j] /= d;
    }
}

double max_elem(double a[][M], int d, int m, int *r, int *c) {
    double mx = a[0][0];
    *r = *c = 0;
    for (int i = 0; i < d; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] > mx) { mx = a[i][j]; *r = i; *c = j; }
    return mx;
}

void report(double a[][M], int d, int m) {
    double t[38], avg[M], sum = 0;
    int r, c;
    row_totals(a, d, m, t);
    col_avgs(a, d, m, avg);
    double mx = max_elem(a, d, m, &r, &c);
    for (int i = 0; i < d; i++) sum += t[i];
    double limit = 0.6 * sum / d;          /* 60% of state average */

    printf("Row totals:");
    for (int i = 0; i < d; i++) printf(" %.0f", t[i]);
    printf("\nColumn averages:");
    for (int j = 0; j < m; j++) printf(" %.2f", avg[j]);
    printf("\nMax = %.0f at district %d, month %d\n", mx, r + 1, c + 1);
    printf("Limit = %.2f, deficient districts:", limit);
    for (int i = 0; i < d; i++)
```

```
        if (t[i] < limit) printf(" %d", i + 1);
    printf("\n");
}

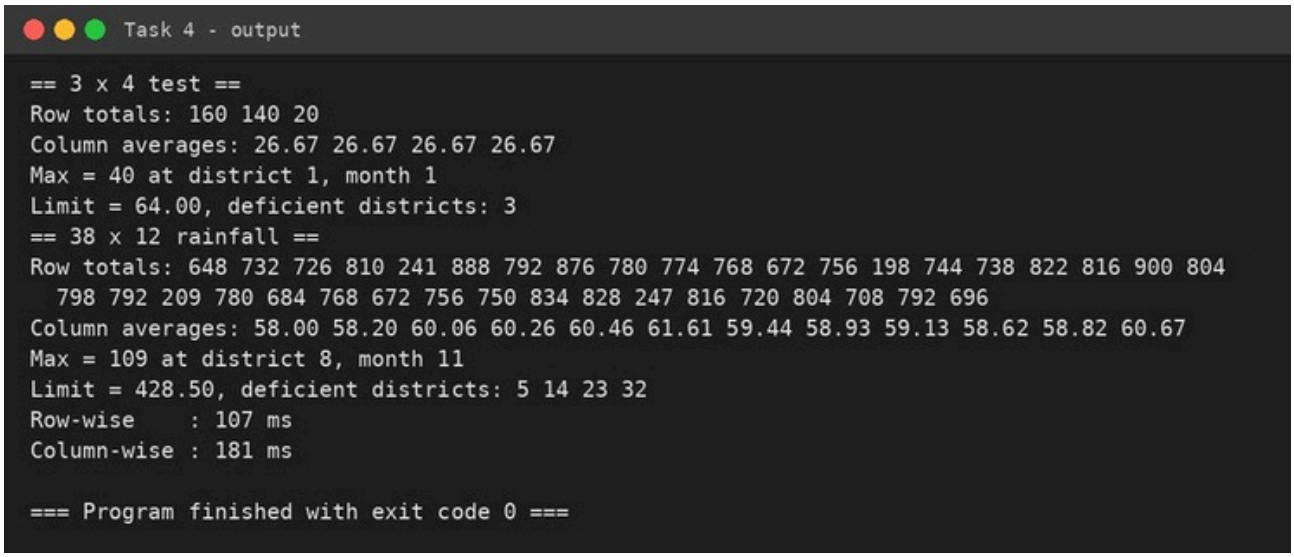
static int big[6000][6000];

int main() {
    double small[3][M] = {{40,40,40,40}, {35,35,35,35}, {5,5,5,5}};
    printf("== 3 x 4 test ==\n");
    report(small, 3, 4);

    double rain[38][M];
    for (int i=  0;i<38; i++)
        for (int j = 0; j < M; j++) {
            rain[i][j]=  20 + (i * 7 +j  * 13) % 90;
            if(i%9==     4) rain[i][j] *= 0.3;      /* dry districts */
        }
    printf("==38x12rainfall ==\n");
    report(rain, 38, M);

    long s = 0;
    clock_t t = clock();
    for (int  i=0;i<6000; i++)
    j=  fporrin(tifn(t"Row-0s0; j++) s += big[i][j];
    wise     t     =    :%.0f  ms\n", (clock() - t) * 1000.0 / CLOCKS_PER_SEC);
    clock();
    for (int  j=0;j<6000; j++)
        for (inti=   0;i<   6000; i++) s += big[i][j];
    printf("Column-wise:%.0f   ms\n", (clock() - t) * 1000.0 / CLOCKS_PER_SEC);
    return s;
}
```

Output Screenshot



Analysis / Discussion

- **Hand check (3 x 4):** row totals 160, 140, 20; each column average = (40+35+5)/3 = 26.67; maximum 40 at district 1, month 1. State average = 320/3 = 106.67, 60% = 64, so only district 3 (total 20) is deficient. The program gives the same.
- **38 x 12 data:** wettest cell is 109 mm (district 8, month 11); deficient districts are 5, 14, 23 and 32.
- **Traversal time:** on a 6000 x 6000 array row-wise took 107 ms and column-wise 181 ms in my run (exact times depend on the computer). Both do the same additions; row-wise is faster because it reads consecutive memory, while column-wise keeps missing the cache.
- **3-D array** rain[year][district][month]: address = base + ((y x 38 + d) x 12 + m) x size; the innermost loop should be the last index (month).
- A 2-D array is better than 38 separate arrays: one name, one loop for all districts, one block of memory.

Result

Total,average, maximum and deficient-district functions were written and verified on a 3 x 4 array. Row-wise traversal is faster because C stores arrays row by row.

TASK 5 - Processing Text Without the Library

Task / Problem Statement

AnSMS feedbackanalyser must count the words in a message, convert it to upper case, reverse it and check whether a short code is a palindrome - first without string.h and then using it.

Aim

Toprocess character arrays with loops (without string.h) and compare the results with the string.h versions.

Concept / Theory

ACstring isachararray that ends with '\0'; every loop stops when it reaches it. The length should be found once and stored - writing strlen(s) in a loop condition counts the whole string every time and makes the loop $O(n^2)$. char s[] = "OK" is a modifiable copy, but char *s = "OK" points to a literal that must not be changed.

Procedure / Algorithm

1Wordcount: whena non-space character appears and we are not inside a word, count one word; a space ends the word. This handles extra spaces.

2 Upper case: replace each character by toupper(c) until '\0'.

3 Reverse: find the length once, then swap s[i] and s[n-1-i] while i < j.

4 Palindrome: move two pointers inwards, skip non-letters, compare ignoring case; return 0 at the first mismatch.

5Write the same functions again with strlen, strcpy and strcmp and compare.

Program / Source Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* ---------- without string.h ---------- */
int word_count(char s[]) {
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (isspace(s[i])) inside = 0;
        else if (!inside) { count++; inside = 1; }
    }
    return count;
}

void to_upper(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) s[i] = toupper(s[i]);
}

voidreverse(chars[]){
    int n = 0;
    while(s[n]!='\0')    n++;               /* length found once */
    for (int i=  0,j=n-     1; i < j; i++,j--) {
        char t=  s[i];s[i]  = s[j]; s[j] = t;
    }
}

int is_palindrome(char s[]) {
    int l = 0, r = 0;
    while (s[r] != '\0') r++;
    r--;
    while (l < r) {
        while (l < r && !isalpha(s[l])) l++;
        while (l < r && !isalpha(s[r])) r--;
        if (tolower(s[l]) != tolower(s[r])) return 0;
        l++; r--;
    }
    return 1;
}

/* ---------- with string.h ---------- */
void lib_reverse(char s[]) {
    char t[200];
    int n = strlen(s);
    for (int i = 0; i < n; i++) t[i] = s[n - 1 - i];
```

```
    t[n] = '\0';
    strcpy(s, t);
}

int lib_palindrome(char s[]) {
    char a[200], b[200];
    int k = 0;
    for (int i = 0; s[i]; i++)
        if (isalpha(s[i])) a[k++] = tolower(s[i]);
    a[k] = '\0';
    strcpy(b, a);
    lib_reverse(b);
    return strcmp(a, b) == 0;
}

int main() {
    char a[] = "Good teaching!", b[] = "Good teaching!";

    printf("Words: %d\n", word_count("  Meet at the library at 5 PM "));
    printf("Palindrome: %d %d\n", is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));
    printf("Empty: %d words | Tab: %d words\n", word_count(""), word_count("Hello\tWorld"));

    to_upper(a);
    printf("Upper : %s\n", a);
    reverse(a);
    lib_reverse(b);
    printf("Reverse: %s | %s\n", a, b);
    printf("Library palindrome: %d %d\n", lib_palindrome("Madam, I'm Adam"), lib_palindrome("Hello"));
    return 0;
}
```

Output Screenshot



Analysis / Discussion

- Outputmatches the expected result: Words: 7 and palindrome 1 0.
- Extra spaces do not change the word count, an empty message gives 0 words, and punctuation is ignored by the palindrome check. The string.h versions give the same results as my own functions.
- isspace() treats a tab as a separator, so "Hello\tWorld" gives 2 words. A version that tests only for ' ' would count 1.
- **Safety:** strcpy does not check the size of the destination, so a long text copied into a small array overflows it; use strncpy/snprintf and fgets. gets() was removed from C11 for the same reason.
- Computing the length once keeps loops O(n). Loop reversal uses O(1) extra space; recursive reversal uses O(n) stack space.
- **Conclusion:** write your own function to learn or for special behaviour; use string.h for normal work because it is tested, fast and short.

Result

Wordcount, upper case, reversal and palindrome check were done with and without string.h and gave the same results.

TASK 6 - A Directory of Names: Sorting and Searching Strings

Task / Problem Statement

Thehostelwardenkeeps 500resident names in names[500][30]. The list is needed in alphabetical order, with search by name and a report of duplicate entries.

Aim

Tosort and search an array of strings using strcmp and strcpy, and compare sorting by copying names with sorting an array of indices.

Concept / Theory

Strings cannot becompared with ==; strcmp(a,b) returns negative, zero or positive by comparing character codes. Capital letters (65-90) have smaller codes than small letters (97-122), so "Apple" comes before "apple". Swapping names needs three strcpy calls, while sorting an index array moves only integers. After sorting, equal names are neighbours, so duplicates are found by comparing each name with the previous one.

Procedure / Algorithm

1
2
3 Fillnames[500][30] (testnames are generated; with real data read each with fgets).
4 Insertion sort with strcmp, moving larger names down with strcpy (count the copies).
5 Insertion sort on idx[] using strcmp(names[idx[a]], names[idx[b]]); only integers move.
Binary search on the sorted names with strcmp, counting comparisons.
Scan the sorted list and count every name equal to its predecessor.

Program / Source Code

```
#include <stdio.h>
#include <string.h>
#define N 500
#define L 30

char names[N][L];
int idx[N];

int main() {
    char *first[] = {"Ravi", "Anu", "Meena", "Kiran", "Sita", "Arun", "Divya", "Hari"};
    char *last[] = {"Kumar", "Rao", "Das", "Nair", "Iyer", "Reddy"};
    for (int i = 0; i < N; i++)
            sprintf(names[i], "%s %s%d", first[(i * 7) % 8], last[(i * 5) % 6], (i * 37) % 300);
    for (int i = 0; i < N; i += 50) strcpy(names[i + 1], names[i]);       /* repeated entries */

    /* index sort: only integers move, names stay where they are */
    long moves = 0;
    for (int i = 0; i < N; i++) idx[i] = i;
    for (int i = 1; i < N; i++) {
        int key = idx[i], j = i - 1;
        while (j >= 0 && strcmp(names[idx[j]], names[key]) > 0) { idx[j + 1] = idx[j]; moves++; j--; }
        idx[j + 1] = key;
    }

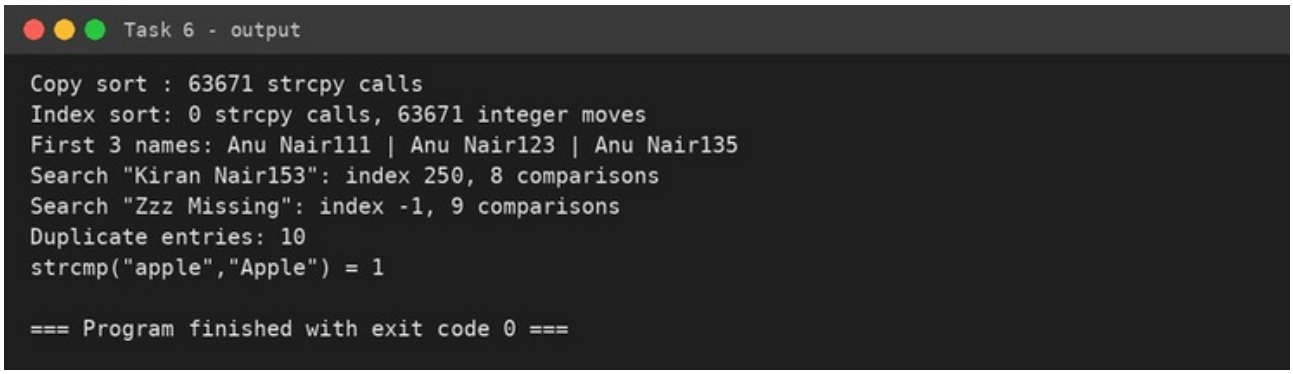
    /* copy sort: names are moved with strcpy */
    long copies = 0;
    char key[L];
    for (int i = 1; i < N; i++) {
        strcpy(key, names[i]);
        int j = i - 1;
        while (j >= 0 && strcmp(names[j], key) > 0) { strcpy(names[j + 1], names[j]); copies++; j--; }
        strcpy(names[j + 1], key);
    }
    printf("Copy sort : %ld strcpy calls\n", copies);
    printf("Index sort: 0 strcpy calls, %ld integer moves\n", moves);
    printf("First 3 names: %s | %s | %s\n", names[0], names[1], names[2]);

    /* binary search */
    char *find[] = {names[250], "Zzz Missing"};
    for (int k = 0; k < 2; k++) {
        int lo = 0, hi = N - 1, steps = 0, pos = -1;
        while (lo <= hi) {
```

```
            int mid = (lo + hi) / 2;
            steps++;
            int c = strcmp(names[mid], find[k]);
            if (c == 0) { pos = mid; break; }
            if (c < 0) lo = mid + 1; else hi = mid - 1;
        }
        printf("Search \"%s\": index %d, %d comparisons\n", find[k], pos, steps);
    }

    /* duplicates are neighbours after sorting */
    int dups = 0;
    for (int i = 1; i < N; i++)
        if (strcmp(names[i], names[i - 1]) == 0) dups++;
    printf("Duplicate entries: %d\n", dups);
    printf("strcmp(\"apple\",\"Apple\") = %d\n", strcmp("apple", "Apple"));
    return 0;
}
```

Output Screenshot



Analysis / Discussion

- **Copyvs index:**thecopy sort needed 63,671 strcpy calls; the index sort needed none and moved 63,671 integers instead. The number of comparisons is the same, but moving integers is much cheaper than copying up to 30 characters.
- **Binary search** among 500 names: 8 comparisons for a present name and 9 for a missing one (about $\log_2 500$).
- **Duplicates:** 10 repeated entries were found by one pass over the sorted list.
- strcmp("apple","Apple") is positive because 'a' (97) > 'A' (65). For a case-insensitive comparison convert both letters with tolower first.
- names[30] holds at most 29 characters plus '\0'; a 30-letter name is cut and needs a larger array or a length check.
- **Conclusion:** sorting indices (or pointers) saves copying time; the cost is a little extra memory and one extra lookup per comparison.

Result

Nameswere sorted, searched and checked for duplicates. Sorting an index array avoids the cost of copying characters.

TASK 7 - Election Counting Program

Task / Problem Statement

A studentcouncilelection hasup to 12 candidates and up to 5,000 votes, each vote being a candidate number. Entries 0, negative, or above the number of candidates are invalid. Report valid and invalid counts, votes per candidate in descending order, and the winner or a tie.

Aim

Todesign a vote counting program from the requirements using a count array.

Concept / Theory

Input: numberof candidates c and the votes. **Output:** valid/invalid counts, sorted results, winner or tie. **Rule:** a vote is valid only if 1 <= vote <= c. **Arrays:** count[13] (index = candidate number) and order[12] (candidate numbers sorted by votes). **Checks:** valid + invalid = n, and a tie exists if the highest count occurs more than once.

Procedure / Algorithm

1. Set count[1..c] = 0 and invalid = 0.
2. For every vote: if 1 <= v <= c then count[v]++, else invalid++ (one pass, no searching).
3. Selection sort order[] by count, highest first (only 12 items).
4. Count how many candidates have the highest count: more than one means a tie, otherwise the first one is the winner.
5. Print valid = n - invalid, invalid, the sorted results and the winner or tie.

Program / Source Code

```
#include <stdio.h>

void election(int v[], int n, int nc) {
    int count[13] = {0}, order[12], invalid = 0;

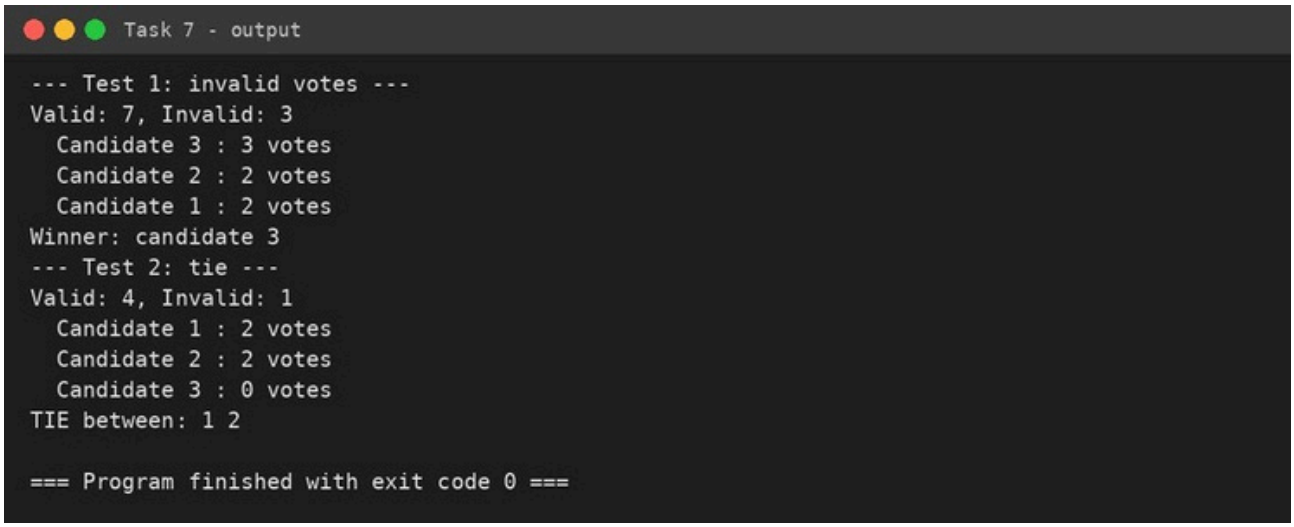
    for (int  i=0;i<n;     i++) {                 /*onepass */
        if (v[i]>=1&& v[i] <= nc) count[v[i]]++;
        else invalid++;
    }
    for (int  i=0;i<nc; i++) order[i] = i + 1;
    for (int      i=0;i<nc     - 1; i++)              /*selection sort, highest first */
        for (intj= i+ 1; j < nc; j++)
            if(count[order[j]] > count[order[i]]) {
                intt=order[i];    order[i] = order[j];order[j] = t;
            }

    printf("Valid: %d, Invalid: %d\n", n - invalid, invalid);
    for (int i = 0; i < nc; i++)
        printf(" Candidate %d : %d votes\n", order[i], count[order[i]]);

    int tied = 0;
    for (int i = 0; i < nc; i++)
        if (count[order[i]] == count[order[0]]) tied++;
    if (tied > 1) {
        printf("TIE between:");
        for (int i = 0; i < tied; i++) printf(" %d", order[i]);
        printf("\n");
    }else
        printf("Winner: candidate %d\n", order[0]);
}

int main() {
    int a[] = {1, 3, 2, 3, 1, 0, 4, 3, 2, -1};
    int b[] = {1, 2, 1, 2, 9};
    printf("--- Test 1: invalid votes ---\n");
    election(a, 10, 3);
    printf("--- Test 2: tie ---\n");
    election(b, 5, 3);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

- **Test 1:** votes 1,3,2,3,1,0,4,3,2,-1 with 3 candidates. Invalid are 0, 4 and -1 (3 votes), valid = 7. Counts: candidate 1 = 2, 2 = 2, 3 = 3, so candidate 3 wins. The program matches.
- **Test 2:** votes 1,2,1,2,9 give candidates 1 and 2 two votes each, so the program reports a tie.
- **Why a count array:** the candidate number is the index, so each vote is counted in O(1) and the whole count is O(v). Searching a list of names for every vote costs O(v x c); sorting all votes costs O(v log v).
- For 500 candidates only the array size changes (count[501]) and the result sorting should use an O(c log c) method such as qsort.
- **Limitation:** the program cannot detect a person voting twice, or a wrong but in-range vote.

Result

Theprogram was designed from the requirements, tested with invalid votes and a tie, and counts all votes in one pass (O(v)).

Test Cases / Validation Table

No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 4: 3 x 4 array {40,40,40,40}, {35,35,35,35},{5,5,5,5}	Totals 160 140 20; col avg 26.67; deficient: district 3	Same	Pass
2	Task 4: maximum of the 3 x 4 array	40 at district 1, month 1	40 at district 1, month 1	Pass
3	Task 5: " Meet at the library at 5 PM "	Words: 7	Words: 7	Pass
4	Task 5: palindrome "Madam, I'm Adam" and "Hello"	1 0	1 0	Pass
5	Task 5: empty message	0 words	0 words	Pass
6	Task 5: "Hello\tWorld" (isspace)	2 words	2 words	Pass
7	Task 6: 500 names, binary search present / missing name	found; -1	index 250 (8 comp.); -1 (9 comp.)	Pass
8	Task 6: strcmp("apple","Apple")	positive	1	Pass
9	Task 7: votes 1,3,2,3,1,0,4,3,2,-1 (3 candidates)	7 valid, 3 invalid, winner 3	7 valid, 3 invalid, winner 3	Pass
10	Task 7: votes 1,2,1,2,9	tie between 1 and 2	TIE between: 1 2	Pass

Final Submission Checklist

- Student andcoursedetails completed
- Task statement, aim, theory and algorithm included
- Program code is readable and runs successfully
- Output screenshot included
- Analysis and result written
- Test cases completed
- GitHub repository link: add if required